SECTION 5: SCALABILITY AND ELASTICITY — HANDLING THE STAMPEDE ⏱ 9 min • MODULE 18

# Part 18: Load Balancing, Gateways, and Reverse Proxies

*Traffic orchestration: understanding Layer 4 vs. Layer 7 routing, Envoy, Kong, and reverse proxy magic.*

🔥 **WAR STORY FROM THE TRENCHES**

A video processing startup had two API endpoints: `/api/v1/user/profile` (which took 5 milliseconds) and `/api/v1/video/transcode` (which took 4 minutes of 100% CPU to transcode an MP4). The infrastructure engineer put all backend servers behind a simple Round-Robin Layer 4 TCP load balancer. At 11:00 AM, five users uploaded heavy 4K videos at the same time. The round-robin algorithm dutifully sent heavy video jobs to Servers 1, 2, 3, 4, and 5. Over the next four minutes, thousands of regular mobile users trying to load their simple 5ms profile pictures were also routed to Servers 1-5, where their requests queued behind the video transcoders and timed out. The entire frontend crashed because the load balancer was blind to application-level paths.

## The Paradigm Shift: From Coder to Architect Mindset

A load balancer is not just a traffic splitter; it is the traffic conductor, security guard, and routing intelligence of your architecture. Moving from dumb Layer 4 (Transport layer: IP/Port) to smart Layer 7 (Application layer: HTTP headers, URLs, cookies) allows you to route traffic based on business urgency, isolate heavy batch workloads, terminate SSL at the perimeter, and inject dynamic telemetry.

## Core Architectural Principles & Mental Models

### Layer 4 vs. Layer 7 Load Balancing

- **Layer 4 (L4 - TCP/UDP, e.g., AWS NLB):** Extremely fast, ultra-low latency, handles millions of packets per second. Operates purely on IP and Port. Cannot inspect HTTP URLs, headers, or cookies.
- **Layer 7 (L7 - HTTP/HTTPS, e.g., NGINX, Envoy, AWS ALB):** Inspects HTTP path, headers, and request body. Enables path-based routing (`/video/*` -> heavy cluster, `/api/*` -> fast cluster), SSL termination, header rewrites, and WebSocket upgrades.

### The API Gateway Pattern (Kong, Envoy, Traefik)

Acts as the single entry point for all clients. Centralizes cross-cutting concerns:

1. Authentication & JWT verification
2. Global Rate Limiting
3. Metrics & OpenTelemetry injection
4. Dynamic canary routing & circuit breaking

## Common Anti-Patterns & Traps to Avoid

- **The Round-Robin Blindspot:** Using simple round-robin routing when request durations vary drastically, leading to server hot-spots. Use 'Least Connections' or 'Peak EWMA' instead.

- **The API Gateway God-Service:** Writing complex business logic, database queries, and custom calculations inside the API Gateway plugins, turning your gateway into a brittle monolithic bottleneck.
- **The Unprotected Backend:** Allowing internal microservices to be directly accessible from the public internet without passing through the gateway.

> 🎯 **THE ARCHITECT'S PLAYBOOK: ACTION STEPS THIS WEEK**
>
> - **Step 1:** Separate your workloads: route long-running heavy compute jobs and fast synchronous queries to dedicated, independently scaled server pools via Layer 7 path routing.
> - **Step 2:** Use 'Least Outstanding Requests' (Least Connections) as your default load-balancing algorithm instead of naive round-robin.
> - **Step 3:** Terminate SSL/TLS at the load balancer or gateway to offload cryptographic compute from your application servers.

## Architectural Takeaways & Cheatsheet

- Layer 4 is about raw speed and packet throughput; Layer 7 is about intelligent routing and security control.
- An API Gateway should handle cross-cutting plumbing, never domain business logic.
- Always route traffic based on workload characteristics to prevent heavy jobs from starving quick queries.

> 🎙️ **UDEMY & NOTEBOOKLM DISCUSSION PROMPT**
>
> **Course Reflection & Hands-on Assignment:** What algorithm does your current load balancer use? If a single backend server starts lagging or experiencing high memory usage, how does your router react?